LOAD BEARING · INTERVIEW PROBLEM GUIDE

Designing a Chat System at L7

One-to-one and group messaging for hundreds of millions of people, the way WhatsApp, Messenger and Slack have to think about it. The guide follows the standard delivery framework, builds the design one requirement at a time, and then adds what a staff or principal answer adds: contracts, numbers that decide something, failure modes, cost and who has to change.

by Chinmay Murugkar

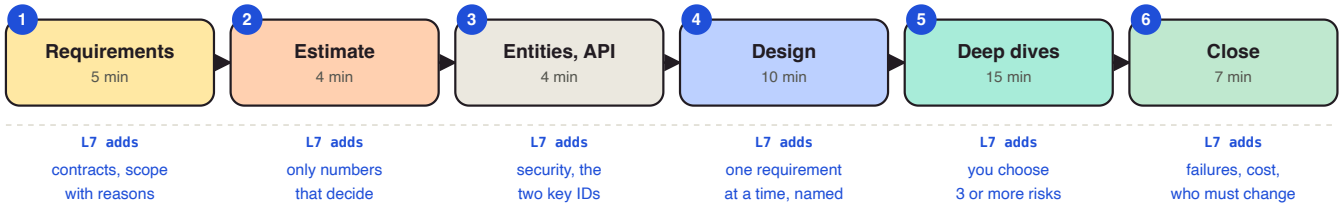
Companion to chinmaymurugkar.com/load-bearing. Numbers marked illustrative are assumptions to state out loud, not benchmarks. Product names are examples you could defend, not the only right answers. Written November 2026.

1. The framework, and what changes at L7
2. Requirements: what's in, what's out, and the contracts
3. Numbers that decide the design
4. Core entities and the API
5. High-level design, one requirement at a time
6. Deep dive: ordering and message IDs
7. Deep dive: delivery guarantees, acks and sync
8. Deep dive: the connection fleet
9. Deep dive: group fanout and storage
10. Deep dive: presence, typing and multiple regions
11. Failures, degradation, evolution and ownership
12. What each level is expected to show
13. Curveballs, a scorecard and corrections
14. Where each concept is taught in Load Bearing

The framework, and what changes at L7

Use the familiar order, because interviewers can follow it: requirements, core entities, API, a high-level design built one requirement at a time, then deep dives. Every level uses the same steps. What changes at L7 is what you put in each one, and how much of the steering you do yourself.

Figure 1 · The delivery framework, with the L7 addition under each step



Illustrative minutes for a 45-minute round. Spend less time on the steps everyone gets right (entities, a REST endpoint) and more on the ones that separate levels (contracts, deep dives you choose, failure).

Step	Typical L5 or L6 content	L7 content
Framing	"Design WhatsApp." Ask about DAU, start listing features.	Say what kind of problem it is: a fleet of long-lived connections plus per-conversation ordering. Storage is the easy part. Propose a scope and say what would change it.
Requirements	Send, receive, groups, online status, "low latency, highly available".	Three or four core features, with the rest named and parked. Non-functional requirements as contracts: delivery latency per path, durability of an acknowledged message, ordering scope, duplicate policy, reconnect behaviour.
Estimation	Messages per second and storage per year.	Concurrent connections and heartbeats per second, because those size the fleet. Storage only to choose the database.
Design	WebSocket servers, a database, a cache, push notifications.	Built one requirement at a time, with a product named on each box and the reason for it. A commit log decides when "sent" is true.
Deep dives	The ones the interviewer asks for.	Chosen by you, from the riskiest parts: ordering, delivery semantics, reconnect storms, group fanout.
Close	Bottlenecks and "add more servers".	Failure walkthrough, a degradation order, an evolution path with triggers, and who has to change. Here that's the mobile team, because clients cause reconnect storms.

The first ninety seconds

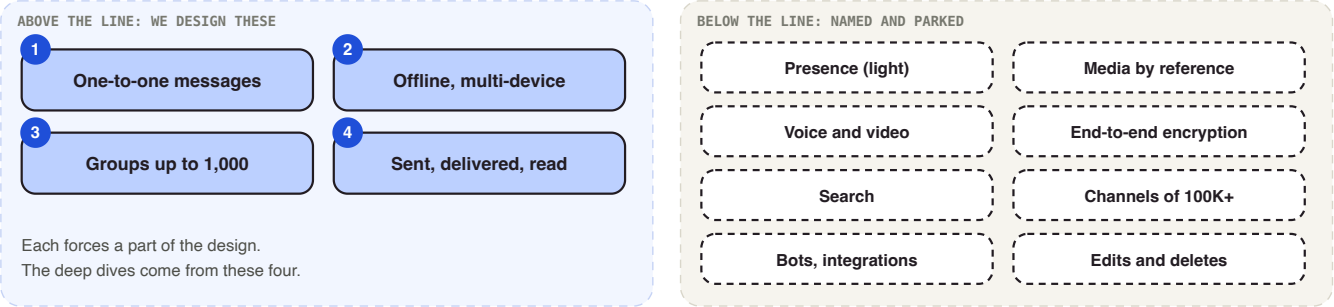
> *“I’ll treat this as a consumer chat app with one-to-one and group conversations, used on phones and laptops, at roughly WhatsApp or Messenger scale. The hard parts are keeping a hundred million or more connections open, getting every member of a conversation to see the same order, and never losing a message the server has acknowledged. Storage is large but well understood. I’ll keep voice, video and end-to-end encryption out of scope unless you want them, because encryption changes what the server is allowed to see. If this is an internal tool for ten thousand employees, most of the fleet work disappears, so tell me if I’ve misread the scope.”*

Why it works: it names the hard parts before drawing anything, parks expensive scope with a reason, and invites a correction without stalling.

What’s in, what’s out

Pick three or four core features and say the rest out loud as “below the line”. Naming them shows product sense. Parking them keeps the design finishable in the time you have.

Figure 2 · Above the line we design. Below the line we name and park.



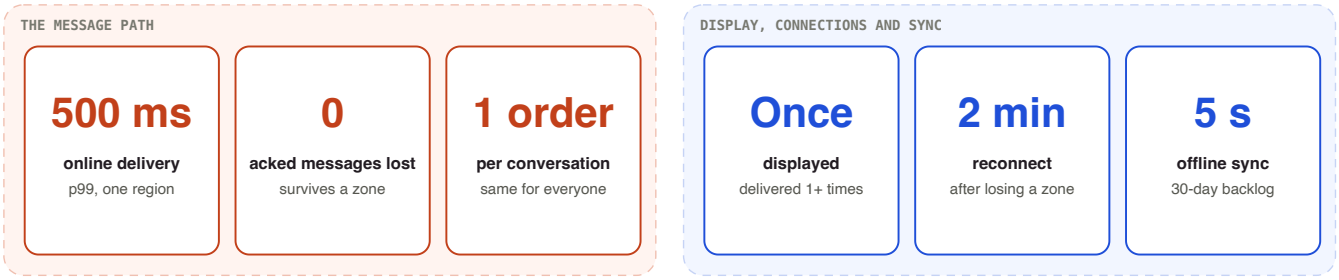
Each parked item has a reason. If the interviewer moves one above the line, the design changes in a way you can predict. Encryption is the biggest: it removes server-side search and moderation, and multiplies fanout by devices.

Core feature	What it means precisely	What it forces in the design
1. One-to-one messages	A sends text to B. B sees it within a second if online.	Persistent connections, a registry of who is connected where, and a durable write before “sent”.
2. Offline and multi-device	B gets everything sent while away, on every device, in order, with no gaps or duplicates.	Per-device sync cursors, push notifications, and history stored on the server.
3. Group conversations	Up to 1,000 members (illustrative). Everyone sees the same order.	An ordering authority per conversation, and a fanout strategy that depends on group size.
4. Receipts	Sent, delivered and read states per message.	Acknowledgements as cursors, not per-message rows, or receipt traffic will exceed message traffic.

Non-functional requirements as contracts

A contract says what’s measured, the target, and the conditions. “Real-time and highly available” is a wish. Split the contracts between the message path and the connection fleet, because they fail differently.

Figure 3 · The headline contracts



Illustrative targets, explained in the table. The ordering and duplicate contracts are the ones most candidates leave out, and they're the ones the deep dives depend on.

Contract	Target (illustrative)	Why that number, and what it costs
Online delivery latency	p99 under 500 ms sender to recipient in one region, under 1 s across regions.	Feels instant in a conversation. Cross-region adds one long network hop that you can't remove.
Durability	Once the sender sees "sent", the message survives losing a zone.	Decides the order of operations: acknowledge after a replicated log write, never before.
Ordering	Every member sees one order within a conversation. No order across conversations.	A single sequencing point per conversation. A global order would be a bottleneck nobody needs.
Duplicates	Delivery is at least once. Display is exactly once.	Client-generated message IDs and deduplication on both ends.
Offline sync	A device returning after 30 days gets every missed message in order, in under 5 s for typical backlogs.	A per-user change log with cursors, not a scan of every conversation.
Send availability	99.99% of sends succeed, measured at the client.	About 4 minutes a month. Rules out single-region designs for the send path.
Reconnect	After losing a zone of gateways, affected clients are back within 2 minutes, without overloading anything.	Forces jittered backoff on clients and headroom on gateways.
Retention and deletion	History kept until the user deletes it. Deletion visible everywhere in 1 minute, purged within 30 days.	Deletion is a write that fans out, plus a background purge for storage and backups.

Numbers that decide the design

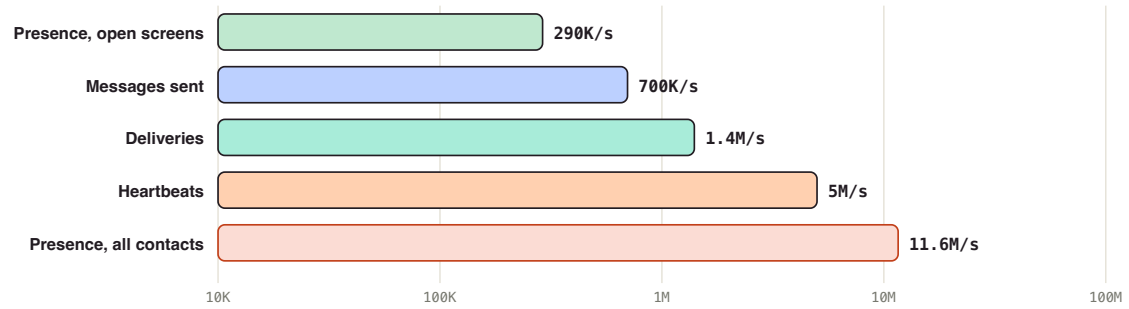
Assume 500 million daily active users, each sending about 40 messages a day, with 30% online at the daily peak. Every number below should end in a decision.

```
Messages: 500M × 40 = 20 billion a day ≈ 230,000 per second on average, about 700,000 per second at peak (3× average)
Deliveries: about 2 recipient devices per message on average (groups and multiple devices) ≈ 1.4M deliveries per second at peak
```

```
Concurrent connections: 30% of 500M = 150 million open sockets
At 100,000 connections per gateway host (illustrative, with TLS) = 1,500 hosts, ≈ 2,250 with headroom to lose one of three zones
Heartbeats every 30 s: 150M ÷ 30 = 5 million per second, seven times the peak message rate
```

```
Storage: 20B × ~250 bytes (text, IDs, metadata) ≈ 5 TB a day, ≈ 15 TB with three replicas, ≈ 5.5 PB a year
Session registry: 150M entries × ~100 bytes ≈ 15 GB, which is small: one Redis cluster
```

Figure 4 · Per-second rates at the daily peak



Log scale. Computed from the assumptions above. Presence is shown both ways: pushed to every contact, and sent only to people who have that chat open (section 10).

WHAT THE NUMBERS SAY

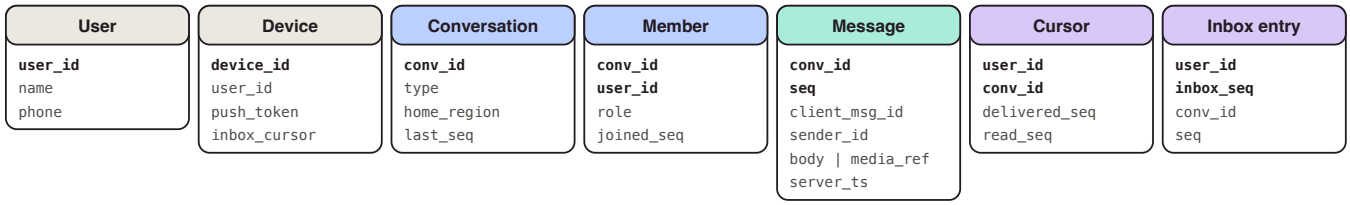
The busiest thing in a chat system isn't messages. It's idle connections saying "still here", and presence updates if you're careless. So heartbeats must be answered by the gateway itself, never by a backend. The fleet is sized by connections and memory, not by message throughput. Storage is large but it's an append-only workload with a known partition key, which is exactly what wide-column stores are for.

For scale: WhatsApp wrote in 2012 that a tuned server held more than two million connections. That shows what's possible with heavy tuning. A planning number should be lower and leave room for TLS, deploys and losing a zone.

Core entities and the API

Keep this short in the room: a list of entities and a few calls. Two details are worth a sentence each, the sequence number and the client message ID, because the deep dives hang on them.

Figure 5 · Seven entities, and the keys that matter



Message key = (conv_id, seq). client_msg_id catches retries. Cursors make receipts cheap, the inbox makes sync cheap.

Bold fields are keys. A message is identified by (conversation, sequence). The client message ID is the deduplication key for retries. Cursors and inbox entries are what make sync and receipts cheap.

Calls over the WebSocket

```
→ send {client_msg_id, conversation_id, body | media_ref}
← ack {client_msg_id, seq, server_ts} the message is durable: show “sent”
← deliver {conversation_id, seq, sender_id, body, server_ts}
→ delivered {conversation_id, up_to_seq} · → read {conversation_id, up_to_seq}
← receipt {conversation_id, user_id, delivered_seq, read_seq}
```

Calls over HTTPS

```
GET /sync?inbox_cursor=8812 → changed conversations and new messages since the cursor
GET /conversations/{id}/messages?before_seq=500&limit=50 → older history, page by page
POST /conversations {member_ids} → conversation · POST /media → signed upload URL
```

SECURITY IN THE API

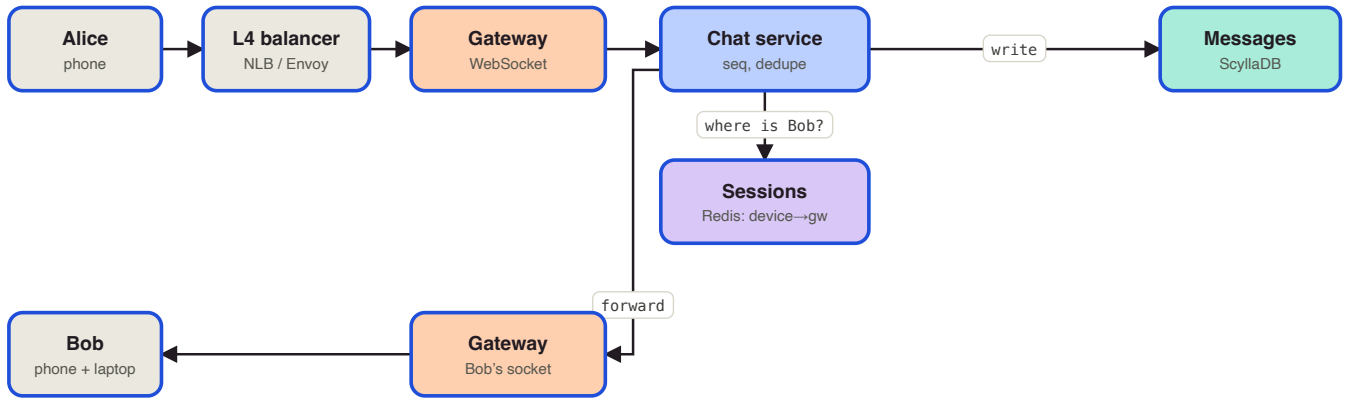
The sender’s user ID comes from the authenticated session, never from the message body. The server sets the timestamp and the sequence number. The client message ID is the only identifier the client chooses, and it’s only used to recognise retries from the same sender.

Build it one requirement at a time

Start with the simplest thing that satisfies requirement 1, then let each new requirement force the next piece. The interviewer sees why each box exists, and you never draw a box you can't justify.

Requirement 1: one-to-one messages, both people online

Figure 6 · Step 1: a direct path



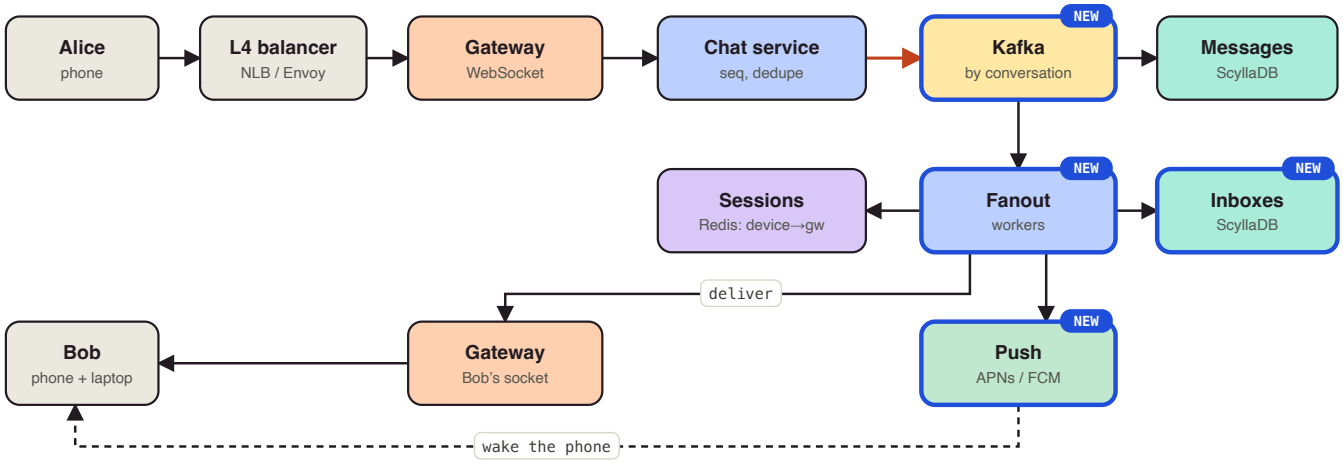
Highlighted boxes are new in this step. Gateways hold WebSockets and are stateful, so the load balancer works at layer 4 and the session registry records which gateway each device is on.

1. Alice's phone keeps a WebSocket open to a gateway through a layer-4 load balancer. The gateway authenticates once, at connect time.
2. The chat service assigns the next sequence number for the conversation, writes the message to the message store, and acknowledges Alice.
3. It looks up Bob's devices in the session registry (Redis) and forwards the message to Bob's gateway, which pushes it down Bob's socket.

What's wrong with it: the chat service does two things in a row, store then forward. If it crashes between them, Bob never gets the message. And if Bob is offline there's nowhere for it to wait.

Requirement 2: offline users and multiple devices

Figure 7 · Step 2: a commit log decides what “sent” means

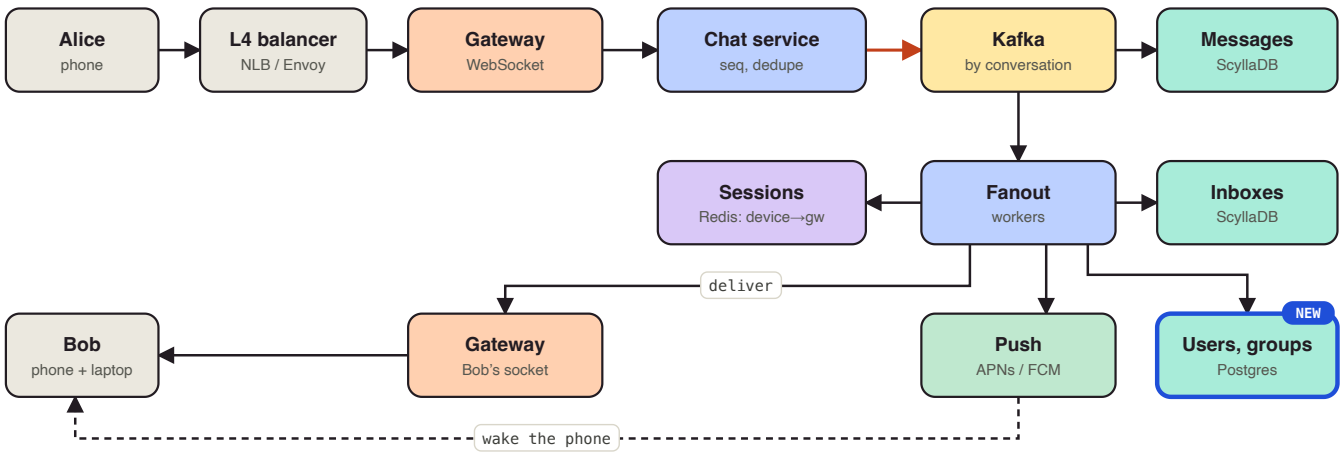


The acknowledgement to Alice now happens after the Kafka write. Two independent consumers persist the message and deliver it, so a crash in either one is retried from the log rather than losing the message.

1. The chat service appends the message to **Kafka**, partitioned by conversation ID, and acknowledges Alice once the write is replicated. That’s the durability promise.
2. A persister writes it to **ScyllaDB** (or Cassandra), keyed by conversation and sequence.
3. **Fanout workers** append a pointer to each recipient’s **inbox**, a per-user change log, then push to any connected device.
4. Devices with no connection get a **push notification** through APNs or FCM. When a device comes back, it asks “what changed since my inbox cursor?” and gets everything in order.

Requirement 3: group conversations

Figure 8 · Step 3: membership drives fanout

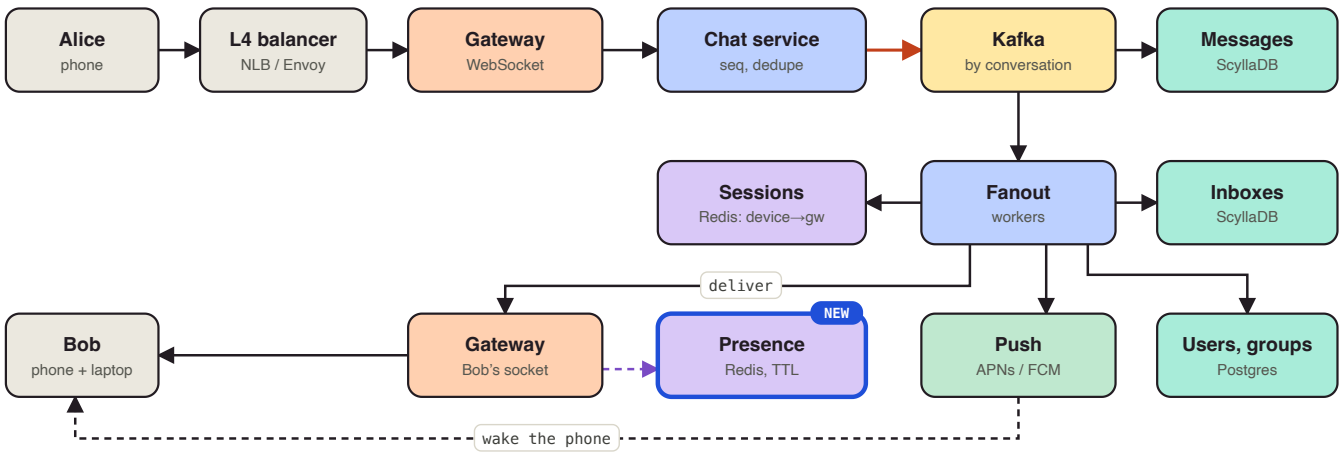


Nothing about ordering changes: a group is a conversation with more members. What changes is how many inboxes one message touches, which section 9 turns into a decision by group size.

Fanout workers read the member list from Postgres, cached in memory, and write one inbox pointer per member. The message itself is stored once. For a 1,000-member group that's 1,000 small writes per message, which is why very large channels use a different model.

Requirement 4: receipts, and presence as a bonus

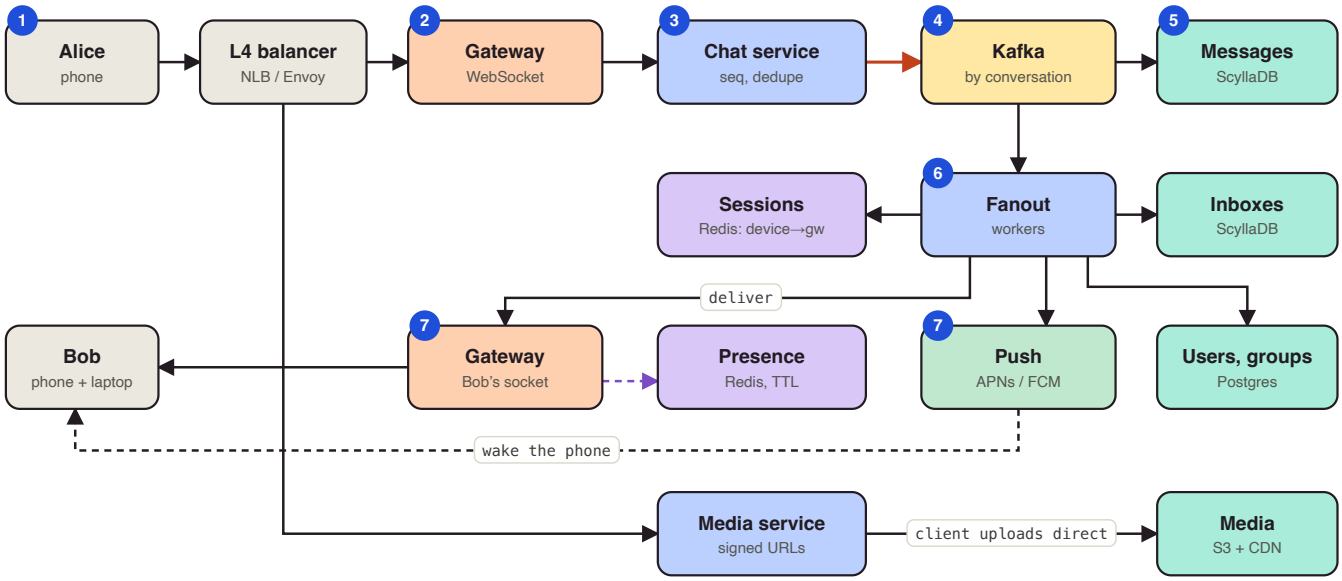
Figure 9 · Step 4: receipts are cursors, presence is a TTL



A device acknowledges "delivered up to sequence 42", not each message. One cursor update covers a burst of messages. Presence is a key with a short time-to-live, refreshed by the gateway while the socket is alive.

The whole design

Figure 10 · The final design, every box named



The numbers follow one message from Alice to Bob, and match the list below. Media never goes through the chat path: the client uploads straight to object storage with a signed URL and sends only a reference. On reconnect, a device calls sync, which reads its inbox and the message store (not drawn).

1. **Client to balancer:** TLS WebSocket through a layer-4 balancer (AWS NLB, or Envoy in TCP mode).
2. **Gateway** (Go, Rust or Erlang): holds the socket, answers heartbeats, enforces per-user rate limits, and registers the device in the session registry.
3. **Chat service:** checks membership, assigns the next sequence number for the conversation, and deduplicates by client message ID.
4. **Kafka**, partitioned by conversation: the acknowledgement to the sender happens after this replicated write.
5. **Persister to ScyllaDB:** partition key (conversation, time bucket), sorted by sequence.
6. **Fanout workers:** read members, append to each recipient's inbox, look up live devices in Redis.
7. **Delivery:** through the recipient's gateway if connected, otherwise a push notification via APNs or FCM.

The names to say, and why

Role	Name to say	The property you need, and an alternative
Layer-4 balancer	AWS NLB, or Envoy (TCP)	Long-lived connections, no per-request parsing. An L7 balancer works but adds a hop that holds every socket.
Gateway	A custom Go or Erlang service	Millions of mostly idle sockets per fleet, and cheap heartbeats. WhatsApp famously used Erlang. Managed options: AWS API Gateway WebSockets for small scale.

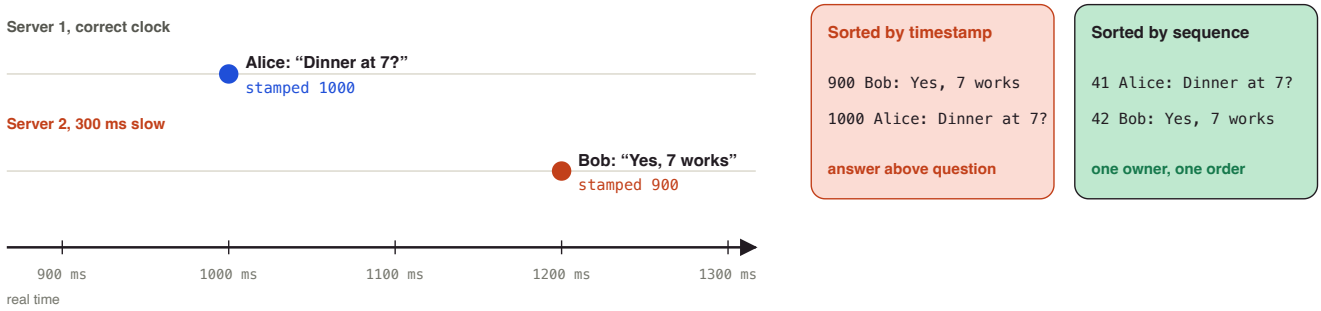
Commit log	Kafka	Replicated, ordered within a partition, readable by several consumers independently. Pulsar or Kinesis are alternatives.
Message store	ScyllaDB or Cassandra	Huge append-only writes on a known partition key, with time-ordered reads. Discord moved from Cassandra to ScyllaDB. HBase is another option.
Session registry, presence	Redis	Fast keys with time-to-live. Losing it is survivable, because devices re-register on their next heartbeat.
Users, groups	Postgres	Small, relational, needs transactions for membership changes. Sharded by user later if needed.
Push	APNs and FCM	The only way to wake a phone that's asleep. Treat them as dependencies with their own failure budget.
Media	S3 plus a CDN	Large blobs stay off the chat path. Signed URLs keep them private.

> *"I'm putting Kafka between the chat service and everything else because I need one durable point where a message becomes 'sent'. After that, storing it and delivering it are two independent consumers that can each fail and retry without losing anything."*

Ordering and message IDs

The contract says every member sees the same order within a conversation. The obvious approach, sorting by timestamp, fails in a way users notice immediately: replies appear above the questions they answer.

Figure 11 · Why timestamps aren’t an order



Illustrative clocks. Server 2 runs 300 ms behind. Bob’s reply is written after Alice’s question but gets an earlier timestamp, so a timestamp sort puts the answer first. A sequence number from one owner can’t be inverted this way.

Option	How it works	Verdict
Client timestamp	The phone stamps the message.	Phone clocks are wrong by minutes, and users can change them. Never use it for order.
Server timestamp	Whichever server receives it stamps it.	Clock skew between servers reorders close messages, as in the figure.
Snowflake-style ID	Time bits plus machine bits plus a counter, unique across the fleet.	Unique and roughly time-ordered, but still skew-prone across machines, and has gaps, so clients can’t detect missing messages.
Sequence per conversation	One owner per conversation hands out 1, 2, 3, and so on.	**Use this.** Total order within the conversation, and gaps reveal missing messages, so clients know to fetch.

Who owns the counter?

Route every send for a conversation to the same place. The cleanest way is to let Kafka’s partition do it: the chat service instance that owns partition *p* handles every conversation that hashes to *p*, and keeps their counters in memory, recovered from the log after a crash. Alternatives are a Redis INCR per conversation (fast, but a second system that must not lose state) or a lightweight transaction in Cassandra (correct, but slow under contention in busy groups).

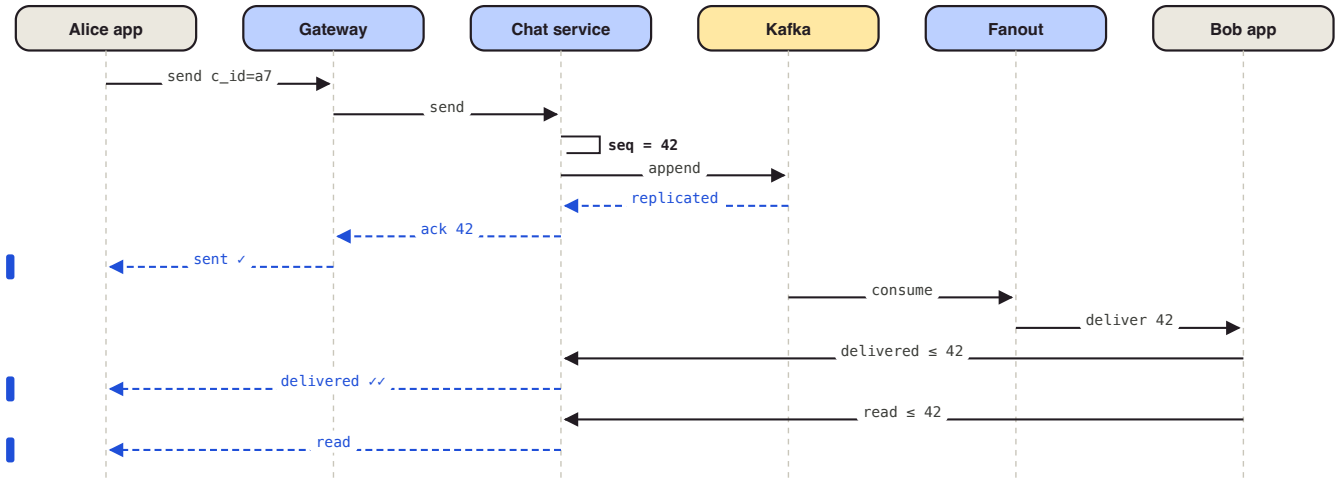
THE L7 ANGLE: SCOPE THE GUARANTEE

Say plainly that there's no order across conversations, and that's fine: nobody can see it. Total order is expensive, so you buy it only where users can tell the difference. That one sentence shows you know what ordering costs.

Delivery guarantees, acknowledgements and sync

Networks drop packets and phones lose signal in elevators. Exactly-once delivery isn't possible over a network that can lose acknowledgements. What you can build is at-least-once delivery with exactly-once display.

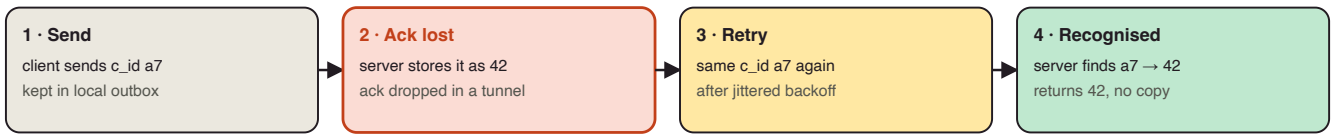
Figure 12 · One message, three acknowledgements



Each acknowledgement is a promise about a different thing. "Sent" means durable on the server. "Delivered" means a device has it. "Read" means a person has seen it. Receipts travel as cursors, so one update can cover many messages.

When an acknowledgement is lost

Figure 13 · A retry that doesn't create a duplicate



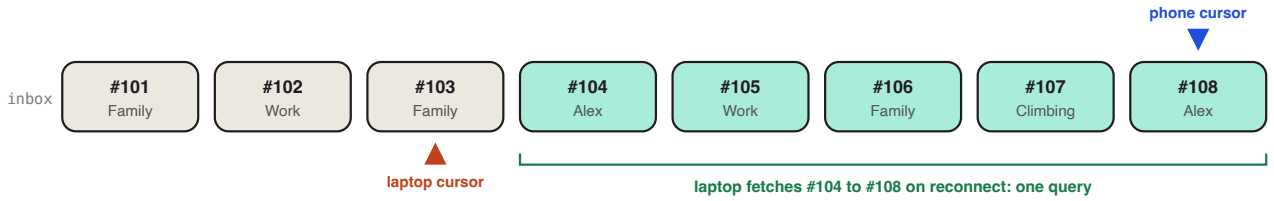
At-least-once on the wire, exactly-once on the screen: the dedupe key is (sender, client message ID).

The client keeps the message in a local outbox until it sees an ack. The server remembers recent client message IDs per sender (for example for 24 hours), so a retry gets back the original sequence number instead of a second copy.

Sync across devices

Each user has an **inbox**: an ordered log of "conversation C has a new message at sequence S". Each device keeps its own cursor into that log. Reconnecting means one query: everything after my cursor. That's cheap even for someone in 500 conversations, because quiet conversations don't appear in the log at all. Facebook described the same shape for Messenger in 2014: a per-user ordered queue of updates, with each device tracking its position.

Figure 14 · One inbox, a cursor per device



Quiet conversations never appear, so sync cost follows activity, not how many chats you're in.

Illustrative positions. The phone is current. The laptop was closed and asks for everything after entry 103. Old inbox entries are compacted once every device has passed them.

GAP DETECTION ON THE CLIENT

Because sequences are dense, a client holding messages 40 and 42 knows 41 is missing and fetches it. This also covers rare cases where a push arrives before the sync that should have preceded it.

The connection fleet

A stateless web fleet can be deployed or scaled at will. A gateway fleet can’t: every host holds a hundred thousand people’s live sockets. Most chat outages that make the news are about this fleet, not the database.

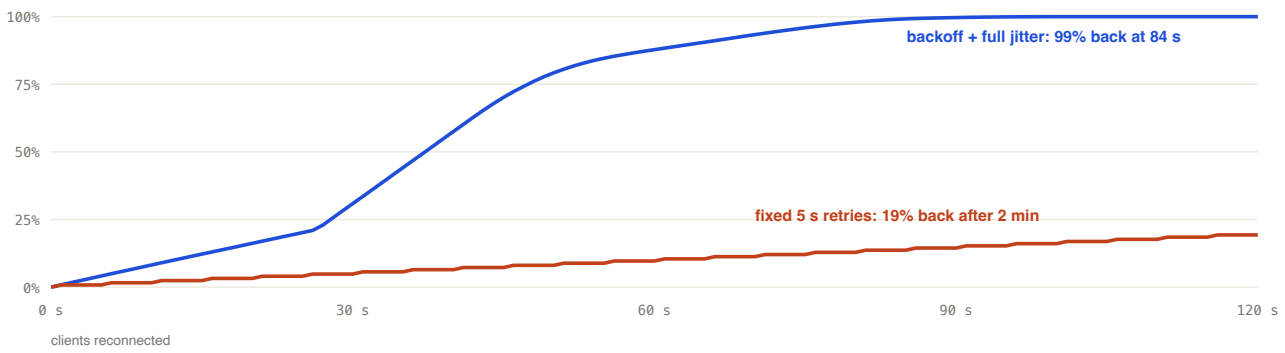
Choosing the transport

	Long polling	Server-sent events	WebSocket
Direction	Server to client, one response per request	Server to client stream	Both ways on one connection
Cost per idle user	A request every timeout	One open connection	One open connection
Use it for	Fallback behind strict proxies	Read-only feeds	**Chat: sends, receipts and typing all go upstream**

Reconnect storms

When a gateway dies or a deploy restarts it, its clients all reconnect at once. If they retry on a fixed schedule they arrive in waves, and each wave can knock over the next gateway, which creates a bigger wave. This is a metastable failure: the system stays broken after the original cause is gone.

Figure 15 · 300,000 clients reconnecting after losing 3 gateways



Illustrative simulation with a seeded random generator. With fixed retries, the first wave is far above what the fleet can accept, so most attempts fail and come back together. With exponential backoff and full jitter, the same clients spread out under the limit and 99% are back in under a minute and a half.

- **Clients:** exponential backoff with full jitter, and a server-sent “retry after” hint. This is code in the mobile app, so the mobile team owns part of the fleet’s stability.
- **Gateways:** admission control (accept at most N new connections per second per host) and headroom to absorb a zone.
- **Deploys:** drain gradually. Tell clients to reconnect over several minutes, instead of killing sockets.

- **Balance:** long-lived connections don’t rebalance on their own. After a deploy, new hosts sit empty while old ones are full. Use connection caps and slow, deliberate draining to even them out.

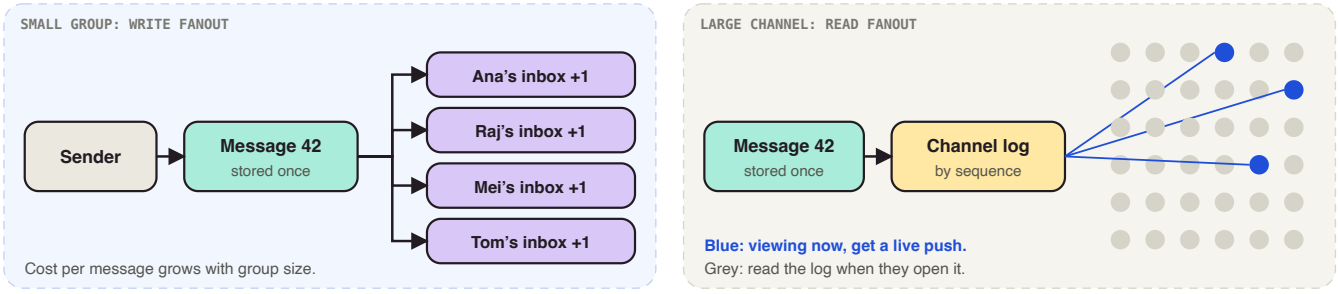
HEARTBEATS AND BATTERY

Mobile networks drop idle connections after a while, so clients send a heartbeat. Too frequent wastes battery and costs millions of packets per second. Too rare and NAT devices silently drop the socket. Pick the interval from measured timeouts per carrier, and let the gateway answer heartbeats without involving any backend.

Group fanout and storage

Write fanout or read fanout

Figure 16 · Small groups copy pointers out. Large channels let members read.



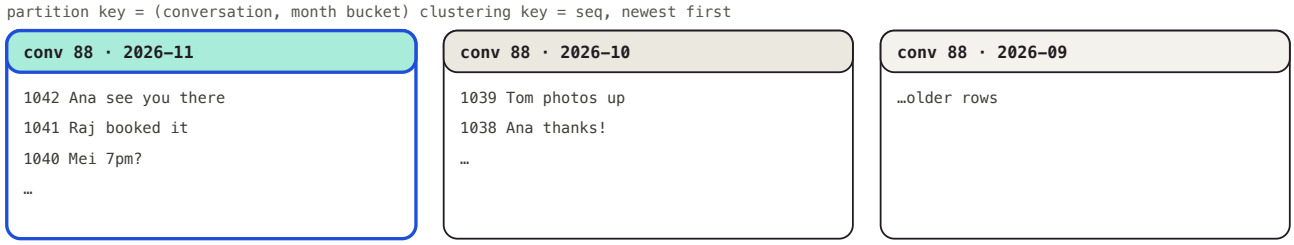
Hybrid rule (illustrative): write fanout up to about 1,000 members, read fanout above.

A pointer is a few dozen bytes, and the message body is stored once in both models. The crossover depends on how many members are online and how often they read. Treat the threshold as a setting you tune, not a constant.

	Write fanout (push to inboxes)	Read fanout (members read the conversation)
Send cost	One inbox write per member.	One write.
Sync cost	One query per device: my inbox.	Check every large channel I'm in.
Fits	One-to-one and groups up to a few hundred or a thousand.	Channels with tens of thousands of members, like Slack or Discord servers.
L7 choice	Hybrid by size. Live delivery to connected members happens in both models. Only the offline catch-up path differs. Large channels also get per-channel rate limits, because one busy channel can otherwise flood every member's connection.	

The message table

Figure 17 · Partition by conversation and time bucket, sort by sequence



"Latest 50 messages" reads the top of one partition. Scrolling back walks to the previous bucket.

Bucketing bounds partition size, so a ten-year-old busy channel doesn't become one enormous partition. Reading recent history touches one or two buckets. Discord described bucketing by channel and time window in 2017.

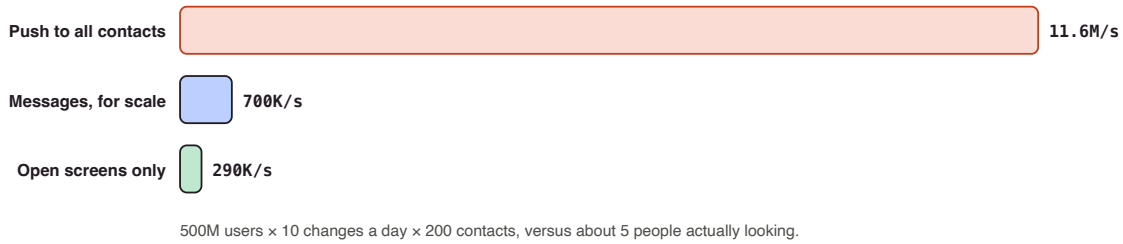
- **Why a wide-column store:** writes are appends, reads are “latest 50 in this conversation”, and the partition key is always known. That’s an LSM-tree’s best case.
- **Hot partitions:** a viral channel concentrates writes on one partition. Smaller time buckets help, and so does rate-limiting the channel.
- **Deletes are writes:** in LSM stores a delete leaves a tombstone, and many tombstones slow reads. Discord’s 2017 post describes exactly this problem. Plan compaction and retention together.

Presence, typing and multiple regions

Presence costs more than messages if you let it

The naive design pushes every status change to every contact. A user with 200 contacts who opens and closes the app 10 times a day creates 4,000 pushes, and across 500 million users that's about 11.6 million per second, more than the messages themselves.

Figure 18 · Presence updates per second, three designs

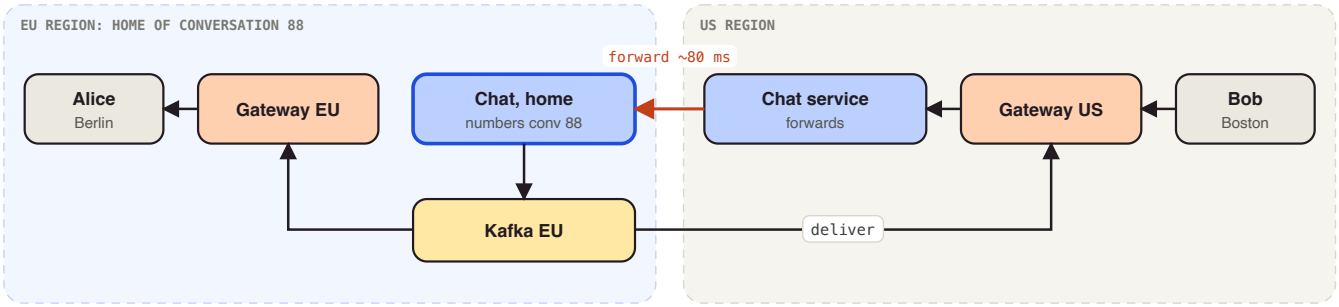


Computed from the assumptions above, with about 5 viewers per change when presence is sent only to open screens. Illustrative, but the order of magnitude is the point.

- **Lazy subscription:** a client subscribes to presence only for conversations on screen, and unsubscribes when it scrolls away.
- **Heartbeat with time-to-live:** the gateway refreshes a Redis key while the socket is alive. If it expires, the user is offline. No explicit "offline" message is needed, so a crashed phone still goes offline.
- **Coarse and delayed:** "last seen a few minutes ago" is cheaper than second-by-second accuracy, and also kinder to privacy.
- **Typing indicators** are never stored, have a short time-to-live, and are the first thing dropped under load.

Multiple regions

Figure 19 · Each conversation has a home region



Users connect to the nearest region. Sends are forwarded to the conversation's home region, where its sequence is assigned, then delivered back out. Only one region numbers a conversation, so ordering holds without global consensus.

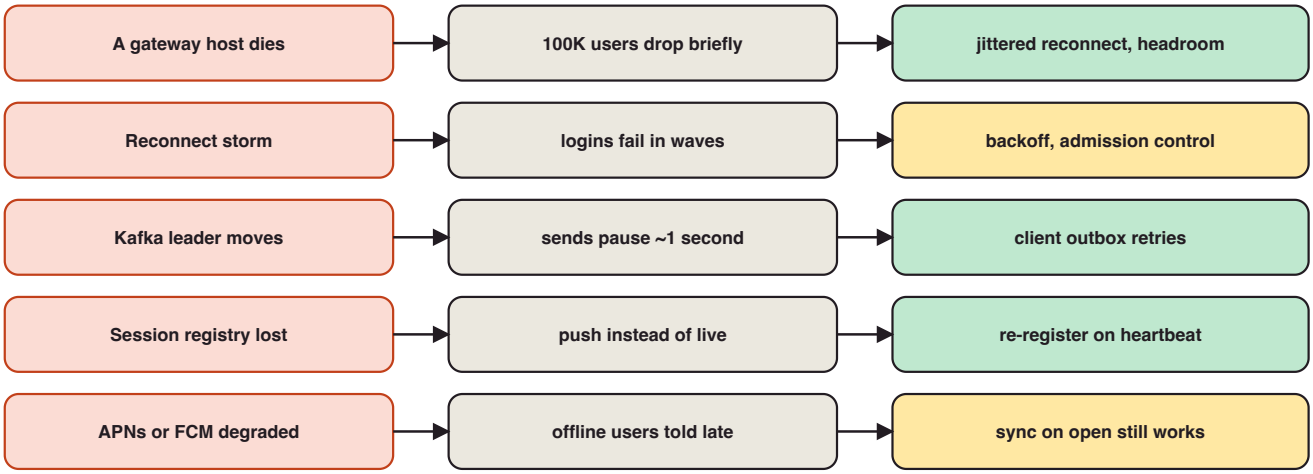
The cost is one cross-region hop for conversations whose members live far apart, which is why the latency contract has a separate cross-region number. If a region fails, its conversations move to another region. That's a planned evacuation with a short pause in sending, not an automatic split-brain.

IF THE INTERVIEWER ADDS END-TO-END ENCRYPTION

The server stores and forwards ciphertext it can't read. Search moves to the device, moderation works from user reports, and push notifications can't show content unless the device decrypts them. With multiple devices, the sender encrypts once per recipient device (Signal's Sesame design), so fanout multiplies by devices. Say these consequences out loud: they change scope, cost and the trust and safety team's tools.

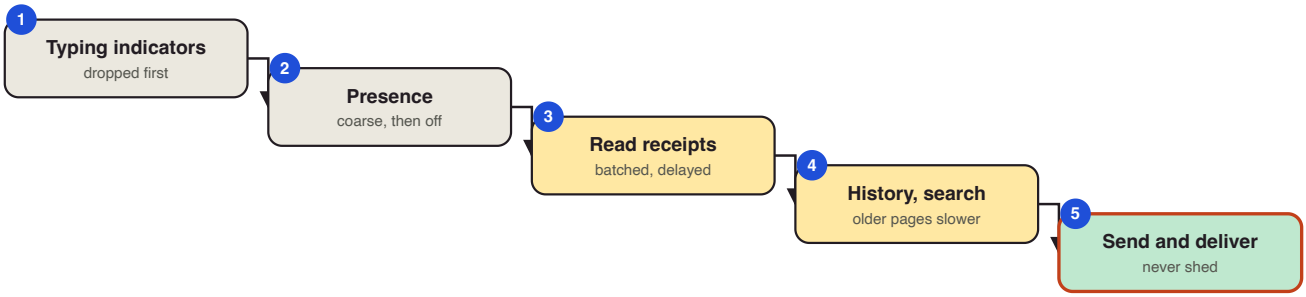
Failures, degradation, evolution and ownership

Figure 20 · What fails, what users see, what contains it



Green: contained by design. Yellow: users notice, but nothing is lost.

Figure 21 · The degradation ladder: what is shed first, and what never is



Each step has a switch built in advance, so on-call can shed load in seconds without a deploy.

Decide this order in advance and build switches for it. During an incident, a system without a plan sheds whatever breaks first, which is often sending.

Figure 22 · Build each step only when its trigger shows up



Each step costs people to run it, so it waits for its trigger.

Cost and ownership

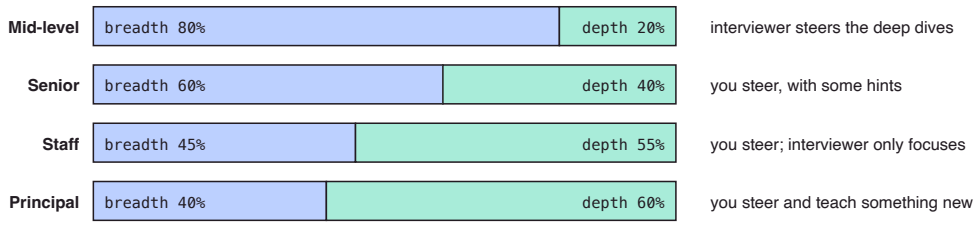
- **Cost per user:** gateways (connections and memory) and storage dominate. Publish cost per thousand monthly users so product teams see what new features cost, especially presence and large channels.

- **The mobile team** owns reconnect behaviour, heartbeat intervals, local outboxes and gap detection. Half of this design runs on the phone.
- **Trust and safety** owns spam and abuse limits at the gateway and on new conversations, and needs reporting tools that still work if encryption arrives.
- **Legal and privacy** own retention and deletion rules per country, which decide where conversations can have their home region.

Question	L5	L6	L7
“Design WhatsApp.”	WebSocket servers, a database, a cache and push notifications. Messages sorted by timestamp.	Adds a session registry, Kafka, a wide-column store, per-device sync and presence with heartbeats.	Frames it as a connection-fleet and ordering problem, writes contracts for durability, order and duplicates, chooses deep dives, plans reconnect storms and fanout by group size, and names who must change: mobile, trust and safety, legal.

What each level is expected to show

Figure 23 · Breadth and depth by level



Rough proportions of interview time, not a scoring rule. The steering share is how much of the conversation the candidate leads without prompting.

Level	The bar for this problem	Common reason for a down-level
Mid-level (L4)	A working design for one-to-one and groups: WebSockets, a registry, a store, push notifications, and a clear API and data model.	Stateless chat servers behind an HTTP balancer, with no idea where a recipient is connected.
Senior (L5)	Moves through the basics quickly, then goes deep on two of: ordering, offline sync, group fanout. Explains tradeoffs with numbers.	Timestamp ordering, or “exactly-once delivery” without saying how.
Staff (L6)	Three or more deep dives, chosen without prompting. Covers reconnect storms and multi-region, and defends product choices from experience.	Never mentions the fleet’s failure modes, or treats presence as free.
Principal (L7)	Everything at staff, plus contracts that drive the design, a degradation order, evolution triggers, cost per user, and the teams that must change. The interviewer learns something.	A perfect architecture with no opinion on what to build first, what to shed, or who owns the phone side.

Curveballs

“A celebrity creates a channel with 2 million members.”

Switch it to read fanout, rate-limit sends to the channel, deliver live messages only to members who are connected and viewing, and give it its own Kafka partitions so it can’t delay normal conversations. It’s really a broadcast product at that size, so ask whether replies should be allowed at all.

“Bob says he never got a message Alice says she sent.”

Walk the acknowledgements: did Alice’s client get “sent” with a sequence number? Is the message in the store? Did Bob’s inbox get the pointer? Did any device acknowledge delivery? Each answer narrows it to one component. The fact that you can answer comes from designing acks as distinct promises.

“How do you edit or delete a message?”

As new events in the same conversation: “message 42 edited” at sequence 57. Clients apply them in order. Purging the original from storage and backups is a separate, slower process with its own deadline.

“Can we drop Kafka and write straight to ScyllaDB?”

Yes, if you acknowledge after the database write and deliver from a change stream or an outbox table. You lose a clean replay point, and delivery depends on database health. It’s a fair tradeoff at smaller scale. Say which you’d pick and why.

“What if this is for 5,000 employees?”

One region, a few gateway hosts, Postgres for messages, and no read-fanout path. Saying the big design is unnecessary here is a strong signal.

“The session registry (Redis) loses its data.”

Delivery to connected devices falls back to push notifications for a minute, and gateways re-register every device on its next heartbeat. Nothing is lost, because messages are in Kafka and inboxes. That’s why the registry can be a cache rather than a database.

Grade your own answer

Area	Weak signal	Strong signal
Framing	Starts drawing WebSocket servers.	Names connections and ordering as the hard parts, and parks encryption with a reason.
Contracts	“Real-time, highly available.”	Latency per path, durability of an ack, ordering scope, duplicate policy, reconnect target.
Numbers	Messages per second and storage only.	Concurrent connections and heartbeats size the fleet. Presence math changes the design.
Ordering	Sort by timestamp.	Sequence per conversation from one owner, with gap detection.

Delivery	“Exactly once.”	At least once plus client IDs, three distinct acks, inbox and cursors.
Fleet	“Add more servers.”	Layer-4 balancing, draining, jittered reconnects, admission control.
Organization	Not mentioned.	Mobile owns reconnect behaviour; trust and safety and legal shape limits and regions.

Corrections to common notes on this problem

Claim	Correction
“Order messages by timestamp.”	Clocks skew between servers and phones. Use a sequence number per conversation from one owner.
“Chat servers are stateless, so any load balancer works.”	Gateways hold live sockets, so they’re stateful. Balance at layer 4, keep a session registry, and drain deliberately.
“Guarantee exactly-once delivery.”	Not possible over a lossy network. Deliver at least once, and deduplicate by client message ID so display is exactly once.
“Send each presence change to every friend.”	That’s millions of updates per second at scale. Subscribe lazily to visible conversations and let keys expire.
“Copy each group message into every member’s inbox.”	Fine for small groups. Large channels need read fanout, or one message becomes thousands of writes.
“Acknowledge once the chat server receives it.”	Acknowledge after a replicated write. Otherwise a crash loses a message the user believes was sent.

Where each concept is taught in Load Bearing

Lessons at chinmaymurugkar.com/load-bearing. `live` means published when this guide was written. `written` means written and coming at the next publish. `planned` means on the index but not yet written.

Concept in this guide	Lesson	Status
Long polling, SSE and WebSockets	WebSockets, SSE and long polling	planned
Running a gateway fleet	Running a fleet of persistent connections	planned
Presence and typing	Presence and typing indicators	planned
Layer-4 balancing of long connections	L4 and L7 load balancing	live
Ordering, clocks and skew	Time, clocks and ordering	live
IDs across a fleet (Snowflake)	Generating IDs across a fleet	live
At-least-once delivery	At-least-once and the exactly-once myth	written
Client message IDs and dedupe	Idempotency from end to end	live
Per-device sync and session guarantees	Read-your-writes, monotonic reads and consistent prefix	live
Kafka as a commit log	Queue or log	written
Write and read fanout	Fanout strategies	written
Consumer lag	Consumer lag as a health signal	written
Partitioning by conversation	Hash, range and directory partitioning	live
Hot channels	Hot keys and skew	live
LSM stores and tombstones	Anatomy of an LSM-tree	written
Compaction	Compaction strategies	written
Retries with jitter	Deadlines, retries and retry storms	live
Reconnect storms as metastable failures	Metastable failures	live
Admission control	Backpressure, shedding and admission control	live
Rate limiting sends	Rate limiting: algorithms and placement	written
Spam and abuse	Abuse and fraud as adversarial systems	written
Cells and blast radius	Cell-based architecture	written
Home regions, active-active	Active-passive and active-active	written
Data residency	Data residency and sovereignty	written

Deletion and privacy	Data deletion and privacy by design	`written`
Requirements as contracts	Requirements as contracts	`live`
Estimation that decides	Estimation that changes a decision	`live`
The full blueprint	Chat and messaging	`planned`

Primary sources

- WhatsApp Blog, “1 million is so 2011” (January 2012): more than two million connections on one tuned server. blog.whatsapp.com
- Facebook Engineering, “Building Mobile-First Infrastructure for Messenger” (2014): a per-user ordered queue of updates, with each device tracking its position. engineering.fb.com
- Discord, “How Discord Stores Billions of Messages” (2017): Cassandra, partitions by channel and time bucket, tombstone problems. discord.com/blog
- Discord, “How Discord Stores Trillions of Messages” (2023): the move to ScyllaDB. discord.com/blog
- Slack Engineering, “Real-time Messaging” (2023): gateway servers for connections, channel servers for channel state. slack.engineering
- Signal documentation: the Double Ratchet and Sesame (multi-device sessions). signal.org/docs
- RFC 6455, The WebSocket Protocol. Brooker, “Exponential Backoff and Jitter”, AWS Architecture Blog (2015).
- Senior-level baselines compared against: Alex Xu, *System Design Interview* Vol. 1, chapter 12, and Hello Interview’s WhatsApp breakdown.